

TRACING MANUAL

Disaster Relief Distribution Network

Kelompok 5

Dokumen ini berisi tracing manual untuk menunjukkan alur kerja struktur data dan algoritma utama pada program. Tracing ini dibuat sebagai lampiran terpisah dari laporan utama.

1. Tracing Min-Heap Prioritas Bantuan

Tujuan

Menunjukkan proses penentuan lokasi prioritas bantuan menggunakan Min-Heap.

Input

Data lokasi berasal dari 'data/nodes.csv'. Lokasi bertipe Gudang tidak dimasukkan ke antrian bantuan karena gudang adalah sumber distribusi.

Rumus Skor Prioritas

$$\text{Skor} = (\text{Kritis} \times 100) + (\text{Risiko} \times 40) + (\text{Kebutuhan} \times 5) + (\text{Populasi} / 100)$$

Semakin tinggi skor, semakin tinggi prioritas bantuan.

Contoh Perhitungan

Posko Porong:

criticalLevel = 5
riskLevel = 5
logisticsNeeded = 35.0
population = 7500

$$\begin{aligned}\text{Skor} &= (5 \times 100) + (5 \times 40) + (35.0 \times 5) + (7500 / 100) \\ \text{Skor} &= 500 + 200 + 175 + 75 \\ \text{Skor} &= 950\end{aligned}$$

Posko Tanggulangin:

criticalLevel = 5
riskLevel = 5
logisticsNeeded = 20.0
population = 3200

$$\begin{aligned}\text{Skor} &= (5 \times 100) + (5 \times 40) + (20.0 \times 5) + (3200 / 100) \\ \text{Skor} &= 500 + 200 + 100 + 32 \\ \text{Skor} &= 832\end{aligned}$$

Desa Renokenongo:

criticalLevel = 5

riskLevel = 5

logisticsNeeded = 8.0

population = 1500

Skor = $(5 \times 100) + (5 \times 40) + (8.0 \times 5) + (1500 / 100)$

Skor = $500 + 200 + 40 + 15$

Skor = 755

Langkah Tracing

Langkah | Proses

- 1 | Program membaca semua lokasi dari graph.
- 2 | Lokasi bertipe Gudang dilewati.
- 3 | Setiap lokasi non-gudang dihitung skor prioritasnya.
- 4 | Lokasi dimasukkan ke Min-Heap berdasarkan skor.
- 5 | Program mengambil 5 lokasi dengan prioritas tertinggi.

Hasil

1. Posko Porong
2. Posko Tanggulangin
3. Desa Renokenongo
4. Desa Lajuk
5. Desa Gempolsari

Kesimpulan

Min-Heap digunakan untuk membantu menentukan lokasi yang harus menerima bantuan lebih dulu berdasarkan kondisi darurat yang dihitung dari beberapa atribut.

2. Tracing Dijkstra dari G01 ke D01

Tujuan

Menunjukkan proses pencarian rute tercepat dari Gudang Pusat Surabaya ke Desa Kedungcangkring.

Input

Asal = G01

Tujuan = D01

Data Awal

Beberapa edge yang relevan:

G01 - P01 = 15.2 km

G01 - P05 = 18.5 km

P01 - P02 = 6.7 km

P02 - D01 = 2.1 km

Langkah Tracing

Langkah | Node Diproses | Jarak Saat Ini | Update Jarak

1 | G01 | 0.0 | P01 = 15.2, P05 = 18.5

2 | P01 | 15.2 | P02 = 21.9

3 | P05 | 18.5 | Tidak menghasilkan rute lebih pendek ke D01

4 | P02 | 21.9 | D01 = 24.0

5 | D01 | 24.0 | Tujuan ditemukan

Perhitungan Rute

G01 -> P01 -> P02 -> D01

$15.2 + 6.7 + 2.1 = 24.0$ km

Hasil

Gudang Pusat Surabaya -> Posko Sidoarjo Kota -> Posko Tanggulangin -> Desa Kedungcangkring

Total jarak = 24.0 km

Kesimpulan

Dijkstra memilih rute dengan total jarak terkecil dari lokasi asal ke lokasi tujuan. Edge yang rusak tidak digunakan dalam proses pencarian rute.

3. Tracing Kruskal MST

Tujuan

Menunjukkan proses pembentukan jaringan distribusi minimum menggunakan Kruskal MST.

Input

Semua edge aktif pada graph digunakan sebagai kandidat jaringan.

Langkah Umum

Langkah | Proses

1 | Program mengambil semua edge aktif.

2 | Edge diurutkan dari bobot terkecil ke terbesar.

3 | Edge terkecil dipilih jika tidak membentuk siklus.

4 | Jika edge membentuk siklus, edge dilewati.

5 | Proses berlanjut sampai seluruh node terhubung dalam jaringan minimum.

Contoh Edge Awal yang Dipilih

Desa Besuki -- Desa Renokenongo == 1.2 km

Desa Besuki -- Posko Porong == 1.5 km

Desa Kalidawir -- Desa Ketapang == 1.5 km
Desa Kedungcangkring -- Desa Pejarakan == 1.8 km
Desa Sentul -- Desa Pamotan == 1.9 km

Hasil

Total Jarak Jaringan: 78.6 km

Kesimpulan

Kruskal MST menghasilkan jaringan distribusi minimum dengan memilih edge berbobot kecil tanpa membentuk siklus. Hasilnya adalah jaringan yang tetap menghubungkan semua lokasi dengan total jarak yang lebih efisien.

4. Tracing Simulasi Jalan Rusak

Tujuan

Menunjukkan perubahan status edge ketika terjadi jalan rusak.

Input

Lokasi 1 = G01
Lokasi 2 = P01
Status = Rusak

Langkah Tracing

Langkah | Proses

- 1 | User memilih menu simulasi jalan rusak.
- 2 | Program menerima ID lokasi 'G01' dan 'P01'.
- 3 | Program mencari edge 'G01 -> P01' dan 'P01 -> G01'.
- 4 | Status kedua edge diubah menjadi tidak aktif.
- 5 | Saat graph ditampilkan, jalur tersebut diberi tanda rusak.

Hasil

Jalur G01 - P01 statusnya menjadi: RUSAK/TERTUTUP

Dampak

- Dijkstra tidak menggunakan jalur yang rusak.
- Jika ada rute alternatif, program mencari rute lain.
- Jika tidak ada rute alternatif, program menampilkan bahwa rute tidak tersedia.

5. Tracing Cek Konektivitas Jaringan

Tujuan

Menunjukkan proses pengecekan apakah seluruh lokasi masih terhubung melalui jalan aktif.

Langkah Tracing

Langkah | Proses

- 1 | Program memilih satu node awal.
- 2 | Program menelusuri semua tetangga melalui edge aktif.
- 3 | Node yang berhasil dikunjungi ditandai sebagai visited.
- 4 | Jika masih ada node yang belum visited, program membuat komponen baru.
- 5 | Jika hanya ada satu komponen, graph dinyatakan terhubung.
- 6 | Jika lebih dari satu komponen, graph dinyatakan terputus.

Hasil Normal

Status: TERHUBUNG

Semua lokasi masih dapat dijangkau melalui jalan aktif.

Total lokasi dalam jaringan aktif: 25

Hasil Jika Terputus

Status: TERPUTUS

Jaringan aktif terpecah menjadi 2 komponen.

Kesimpulan

Tracing konektivitas membantu menunjukkan apakah jaringan distribusi masih dapat menjangkau semua lokasi atau terdapat lokasi yang terisolasi.